Lab 3: Trajectory Control

Midterm exam – Oct 15, 2025

Covering all aspects of class so far (HW 1-4):

- Arduino + arduino code
- Brushed DC motors
- Gears
- Binary representations of data
- Port registers and digital logic
- PID control

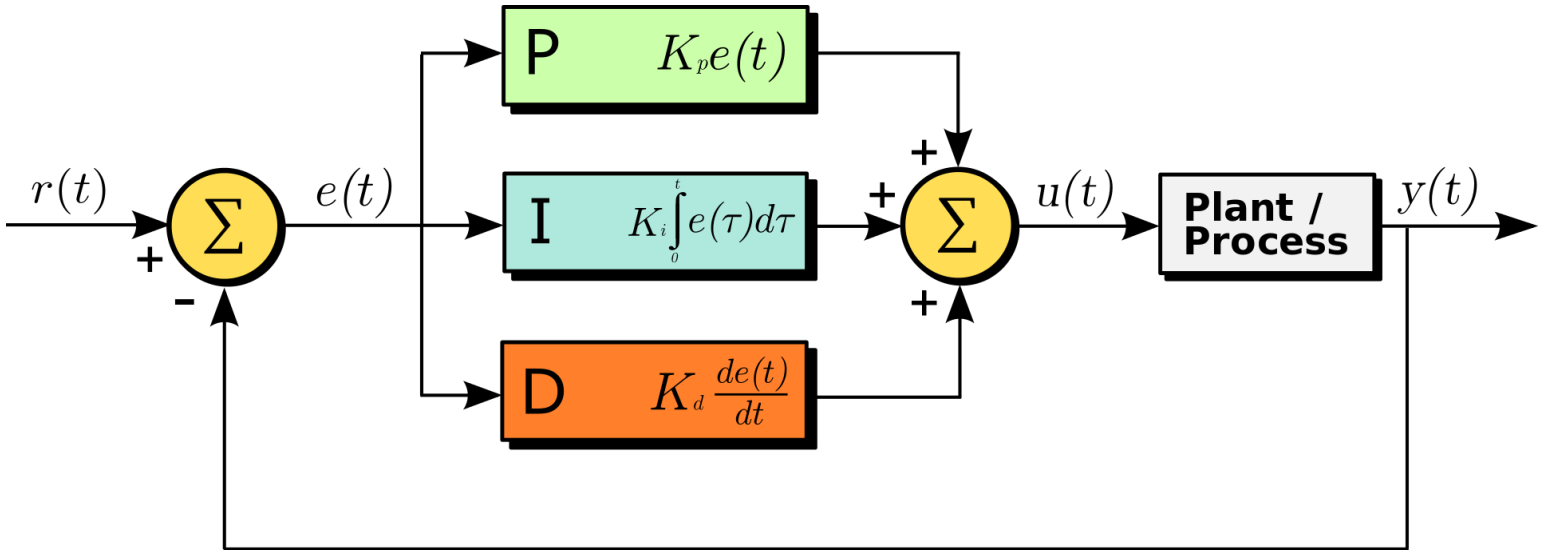
8-10 short problems (nothing long and involved, no diff eq. solving)

PID Controller

P: Proportional

I: Integral

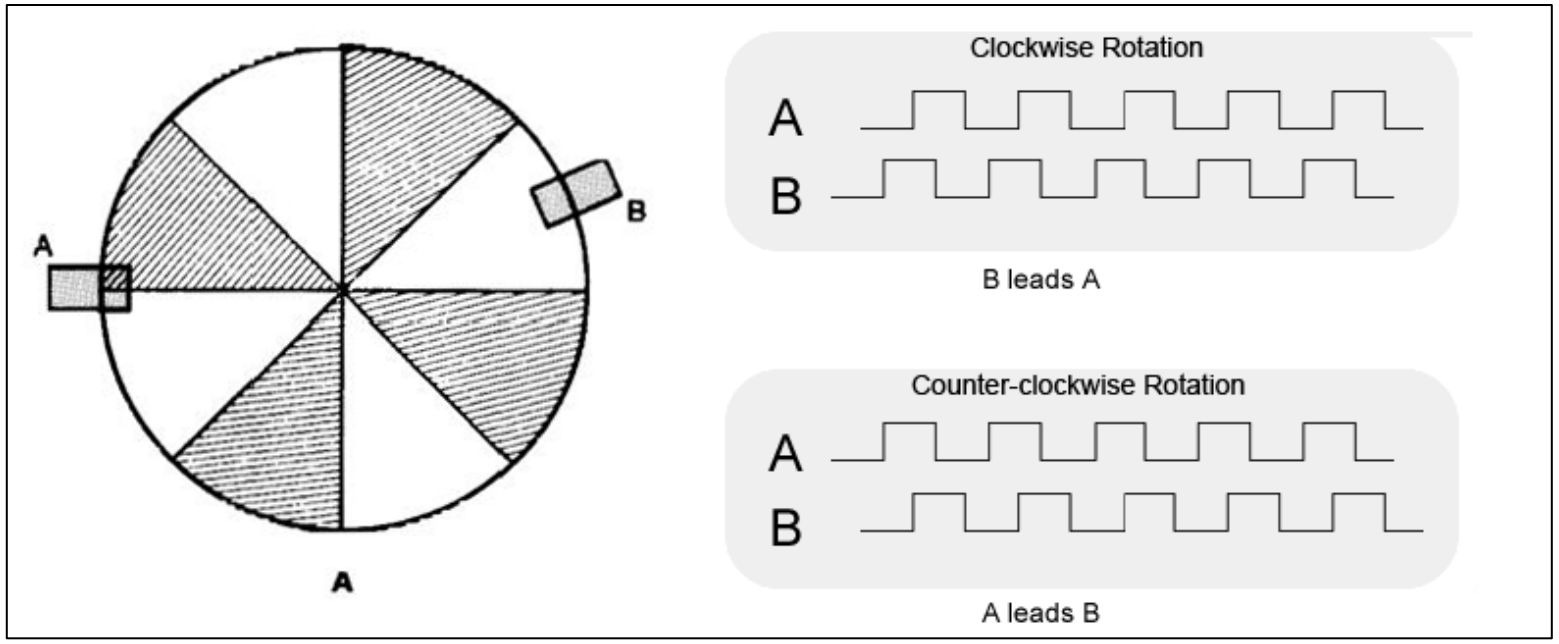
D: Derivative



$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Quadrature encoders

- Counts per line (4)
- Detecting the direction of the motion is based on the offset of the two signals



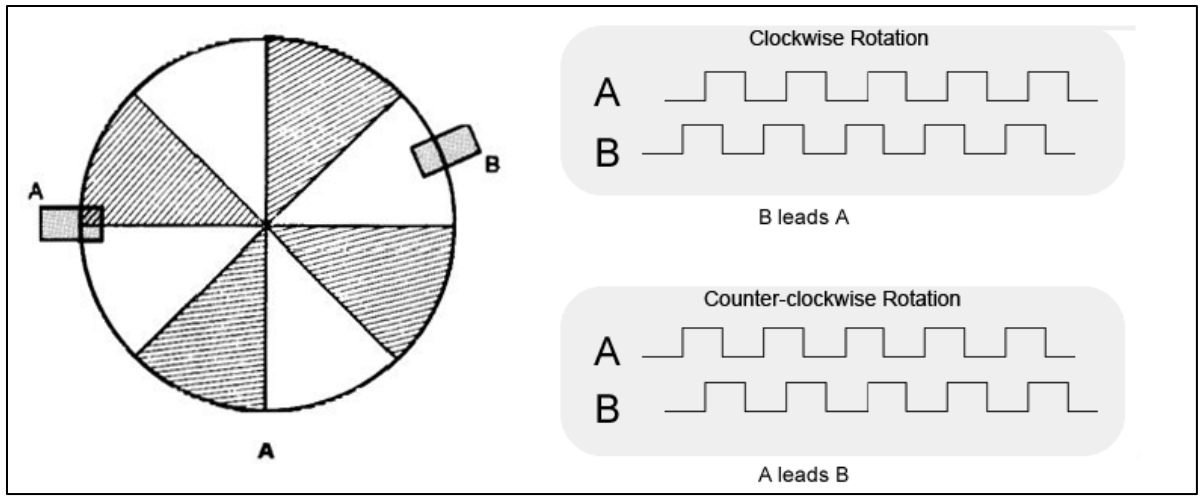
Encoders + conversion from cm to counts

Moving distance of car

$$dist(\text{counts}) = \frac{\text{counts}}{\text{rev}} \cdot \frac{\text{rev}}{\text{cm}} \cdot dist(\text{cm})$$

Counts/revolution or pulses/revolution

1/Wheel circumference



For a wheel with a 6 cm diameter and an encoder resolution of 900 counts per revolution, how many counts are required to travel **10 cm?**

- **Wheel diameter** = 6 cm
- **Wheel circumference** = π × d ≈ 3.14 × 6 ≈ **18.85 cm**
- **Encoder resolution** = 900 counts/rev (i.e., the encoder generates 900 pulses per full wheel rotation)
- **Target travel distance** = 10 cm

Step 1: Calculate revolutions per centimeter

$$\text{rev/cm} = \frac{1}{\text{circumference}} = \frac{1}{18.85} \approx 0.053 \text{ rev/cm}$$

Step 2: Calculate counts per centimeter

$$\text{counts/cm} = \text{counts/rev} \times \text{rev/cm} = 900 \times 0.053 \approx 47.7 \text{ counts/cm}$$

Step 3: Calculate counts for 10 cm

$$\text{counts} = \text{counts/cm} \times \text{distance(cm)} = 47.7 \times 10 \approx 477 \text{ counts}$$

If a wheel has a diameter of 6 cm and the encoder resolution is 900 counts per revolution, how far has the robot traveled if the encoder registers 1000 counts?

Given:

- Wheel diameter = 6 cm
- Wheel circumference = π × d ≈ 18.85 cm
- Encoder resolution = 900 counts/rev
- Observed counts = 1000

Step 1: Counts per cm

$$\text{counts/cm} = \frac{\text{counts/rev}}{\text{circumference}} = \frac{900}{18.85} \approx 47.7 \text{ counts/cm}$$

Step 2: Convert counts to distance

$$\text{distance(cm)} = \frac{\text{counts}}{\text{counts/cm}} = \frac{1000}{47.7} \approx 20.96 \text{ cm}$$

Lab 3 part 1

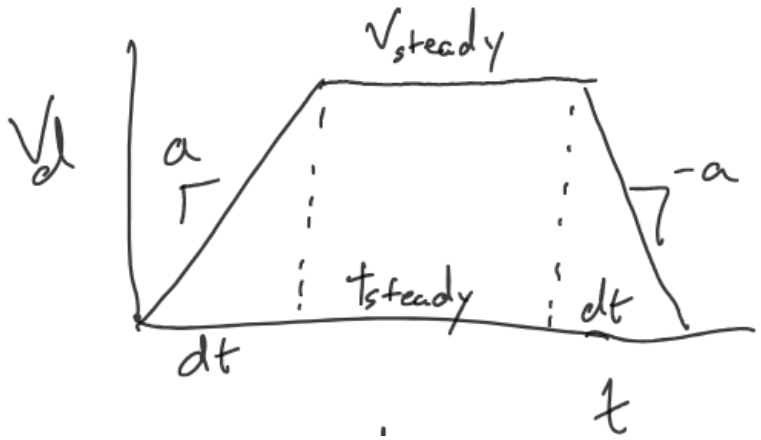
- Connect and test Encoders using the <Encoder.h> package
- Write and *briefly* tune a P controller to follow waypoints using setpoint control
- Follow course using feedback from encoders

Extra credit:

- Fastest time

$$pwm = K_p(X_d - X)$$

Trapezoidal Velocity Profile



common method in robotics and motor control to ensure smooth motion when traveling a set distance.

1. Velocity Profile

- **Acceleration phase**: The velocity increases from 0 with acceleration a, lasting

$$\Delta t = \frac{V_{steady}}{a}$$

- **Constant velocity phase**: The velocity stays at V_{steady} for a duration of t_{steady}.
- **Deceleration phase**: The velocity decreases with $-a$, also lasting Δt.
- Together, these form a trapezoid-shaped velocity curve.

Steady-state time:

$$t_{steady} = t_{total} - 2\Delta t$$

Total displacement (area under the curve):

$$X_{total} = \tfrac{1}{2}\Delta t \cdot V_{steady} \cdot 2 + V_{steady} \cdot t_{steady}$$

$$X_{total} = \Delta t \cdot V_{steady} + V_{steady}(t_{total} - 2\Delta t)$$

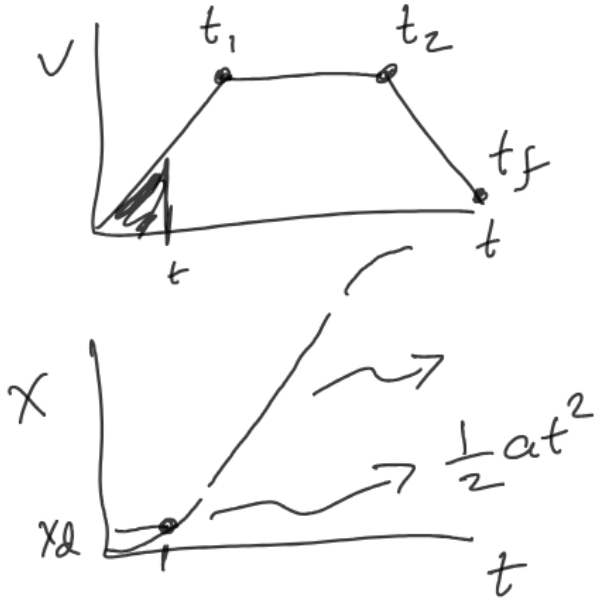
Integral Form

$$X_d = \int_0^{t_{total}} V(t)\, dt$$

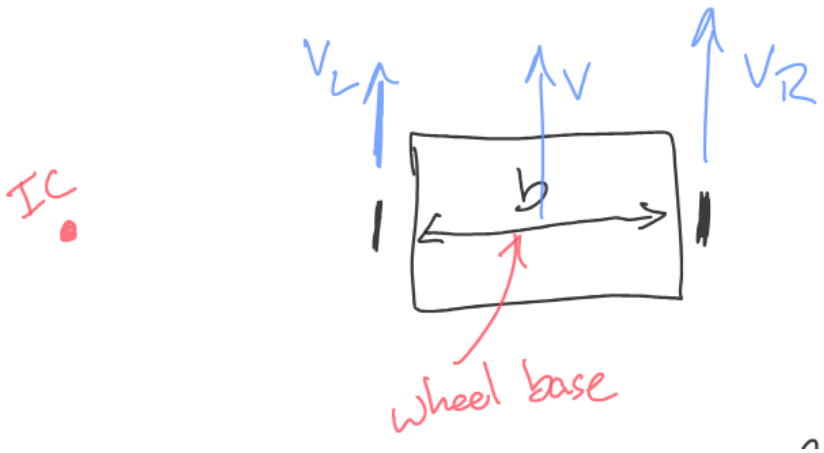
Final relationship

$$t_{total} = \frac{X_{total} - \frac{V_s^2}{a}}{V_s} + 2\Delta t$$

Get position + velocity from trapezoidal profile



Differential Drive Kinematics



kinematics of a differential drive robot (a robot with two independently driven wheels).

1. Diagram

- The robot has two wheels: left wheel velocity V_L and right wheel velocity V_R.
- The center of the robot moves with linear velocity V.
- The distance between the two wheels is the **wheel base** b.
- The red point "IC" stands for the **Instantaneous Center of Curvature**, the point around which the robot rotates when turning.

2. Key Equations

- **Linear velocity (forward speed):**

$$V = \frac{V_L + V_R}{2}$$

The average of left and right wheel speeds.

- **Angular velocity (turning rate):**

$$\omega = \frac{V_R - V_L}{b}$$

The difference in wheel speeds divided by the wheel base.

3. Inverse Relations (given V, ω, find wheel speeds)

- Left wheel velocity:

$$V_L = V - \frac{b}{2}\omega$$

- Right wheel velocity:

$$V_R = V + \frac{b}{2}\omega$$

Lab 3 part 2

- Write **and tune** a PD controller to follow waypoints with desired velocity and desired position
- Write a motion planner to create waypoints in time using a trapezoidal velocity profile

Extra credit:

- Fastest time
- Include integral control to make a full PID controller
- Include angular velocity control based on drift (+ cut corners?)

$$pwm = FF + K_p(X_d - X) + K_d(V_d - V)$$

In control systems, **feed-forward (FF)** means predicting the required control input directly from the desired behavior, instead of waiting for an error to appear.

$$FF = \frac{V_d}{V_{max}} \cdot 255$$

V_d: desired velocity (e.g., in cm/s or counts/s).

V_{max}: maximum motor velocity (when PWM = 255).

255: maximum PWM value for an 8-bit controller.

If the motor's max speed = 100 cm/s, and you want $V_d = 50$ cm/s,

$$FF = \frac{50}{100} \times 255 = 127.5 \approx 128$$

Classes in C++

```
class Controller {
    public:
        float Kp;
        int pwm;
        Controller(float kp) {
            Kp = kp;
        }
        void updatePWM(float err);
};
```

```
Void Controller::updatePWM(float err) {
    pwm = Kp*err;
}
```